CSI v10 — Creating Form Personalizations

Study notes from the Infor official training workbook (129 pages)

Why this guide matters

Unlike the navigation guide, this is **real developer-track material** — it teaches you to actually customize SyteLine/CSI forms in **Design Mode** using **WinStudio**. This is the closest thing to "how the job actually works" out of everything covered so far: IDOs, permissions, components, bindings, validators, event handlers, and FormSync (deployment between environments).

---

1. The data architecture: IDOs and the three-tier model

An **IDO (Intelligent Data Object)** is a business object that transports data between the client and the database, including validation/rules. An IDO consists of a reference to one or more SQL tables, a set of property definitions, a set of zero or more methods, and an optional reference to an IDO Extension Class.

The Mongoose framework gives every IDO standard built-in functions: **LoadCollection** (retrieves rows), **UpdateCollection** (executes insert/update/delete SQL), **GetPropertyInfo** (returns property metadata), **Invoke** (runs a custom method).

Three-tier architecture: | Tier | What happens here | |---|---| | 1. Database | Columns get attributes/properties when added to a table | | 2. Middle tier (IDO) | Defines properties bound to columns, plus calculated properties | | 3. Form (client) | Components bind to IDO properties, inheriting their attributes; you can override/add attributes at the component level |

Practical takeaway: before modifying or adding a component, check if a reusable one already exists — reuse saves time and keeps the app smaller.

---

2. Form editing permissions and scope

Every form/global object change happens at one of four **scope levels**, in order of precedence:

Scope	ScopeType	Who can create/edit it	Who sees it
Vendor	0	Vendor Developer only	Shipped default; overridden by anything below
Site Default	1	Site Developer only	Everyone, unless a group/user version exists
Group	2	Site Developer only	Users whose primary group matches
User	3	Site Developer, Full User, or Basic User (the latter two only for themselves)	Just that one user

Resolution example: if Bob has no personal version, he sees his group's version; if no group version, the site default; if no site default, the vendor default.

Edit permission levels (assigned on the Users form):

Permission	Can do
Vendor Developer (value 4, hidden from dropdown — set manually for diagnostics)	Edits vendor defaults. Logging in as this user **ignores all customizations** — useful to check if a bug is caused by a customization. Never make real changes here.
Site Developer	Full power: create/delete forms at site/group/user scope, customize anything, impersonate any user/group to view their version
Full User	Can fully customize forms but **only their own user version** — can't create/delete forms
Basic User	Simple customizations only (hide, read-only, colors/fonts, rearrange) at user level — can't create/delete forms

Permission	Can do
None	Cannot enter Design Mode at all

Check your current permission level: `Help > About <servername>`.

Explorer folders and who controls them:

- **My Folders** — personal, auto-created on login, fully editable by you
- **User Folders** (Site Developer view only) — mirrors a specific user's My Folders; editable by Site Developer, changes reflect back to that user
- **Master Explorer** — Vendor Developer-created folders + the read-only "All Forms" list
- **Public Folders** — empty by default; Site Developer populates for all users to see

---

3. Editing forms — fonts, colors, images, basic properties

Design Mode basics

Open form → remove filter-in-place → **Design Mode** → confirm editing scope (if Site Developer) → form displays with **Toolbox** (left) for adding components, and **Form/Component/Objects sheets** for properties.

- **Form sheet** = whole-form attributes
- **Component sheet** = individual component attributes
- **Objects sheet** = work with global objects independent of the form

Fonts

Two ways to change: **User Preferences** (`System > View > User Preferences > Runtime Layout tab`) changes the whole theme (Infor or Classic, the two built-in non-deletable themes); or **Form/Component sheet > Appearance** for form/component-specific overrides.

Regenerating a form (saves, closes, reopens in Design Mode) is sometimes needed to see changes — via the Regenerate Form icon or `System > Edit > Regenerate Form`.

Background colors

Solid color (with optional transparency) or linear gradient (two colors, each with its own transparency). Set via Form/Component sheet **or** the **Edit Color dialog** (`System > Edit > Background Color`) for reusable colors.

Background Color field values tell you the source: blank = unset, RGB numbers = literal color, `V(bgc_Form)` = set via form attribute, `V(bgc_componentName)` = set via component attribute, `V(gbgc_colorDefName)` = a reusable global color definition.

Background images

Set via the **Background Image** form property — four attributes: Image, Center, Disable On Pane 0, Disable On Pane 1. Supported formats: BMP, EMF, EXIF, GIF, WMF, ICO, JPG, PNG, TIFF. Images must first be **imported into the forms database** (`System > Edit > Image` in Design Mode) so the path works for every user. A **banner** uses the Static component's Bitmap File Name field instead.

Basic component behaviors

Property	**Effect**
Read Only/Disable = True	Field becomes non-editable
Hidden = True	Component invisible (may need regenerate to see)
Visible When / Enabled When / Required When	Conditional behavior based on rules you define
No Tab Stop	Skips component in tab order
Auto Complete	Combo box shows matching list as user types

Date/time/binary data formatting and **input masks** (e.g. phone number patterns) are also configurable per-component — use mask character `9` for decimal fields that can't be null, defaulting to 0.

Reverting and copying forms

Check if a form is customized: `System > Help > About This Form` (anything other than "Vendor Default" = customized). **Revert** rolls back one level (User→Group→Site→Vendor) via `System > Edit > Revert Form Definition`. **Copy** a form first if you want to keep customizations before reverting (`System > Form > Definition > Copy`).

---

4. Customizing form components

Toolbox component types

Most commonly used: **Button, CheckBox, DateCombo, Edit** (single-line only — use MultiLineEdit for multiple lines), **Combo, Grid, GridColumn** (only creatable via Edit Contained Components), **GroupBox, MenuItem, NoteBook, NotebookTab, RadioButton, Static** (read-only text or images), **ToolbarButton**.

Rarely used: Browser, DropList, FormPage, Graph, HyperLinkButton, List, MultiLineEdit. Tree and User Control are considered too complex for this course's scope.

Adding and aligning

Drag from Toolbox → draw on form → Component sheet appears to set properties. Delete with the Delete key or `System > Edit > Component > Delete Selected`.

Alignment toolbar (only active in Design Mode with 2+ components selected): Align Left/Right/Top/Bottom/Center/Middle, Make Same Width/Height/Size, Space Horizontally/Vertically, Arrange in Column/Row, Bring to Front/Send to Back, Undo/Redo. Components align to whichever one has **solid black handles** (the last one selected).

Tab order

Set via `System > Edit > Component > Tab Order` or the Alignment toolbar button, then click components in desired order (zero-based). Rules: include every component you want in the sequence, select a label's static component right before its paired field, select a container before the components inside it. **Color coding while setting:** blue-on-white = already clicked, white-on-red = last clicked, white-on-blue = not yet clicked. On extended forms, dark-blue-on-light-blue = belongs to base form, can't be changed.

---

5. Inherited attributes — the most conceptually important lesson

This is the inheritance chain that governs how component properties get their defaults, from broadest to most specific:

IDO Property Class → IDO Property → Property Class Extension → Component Class → Component

(broadest) (most specific, always wins)

Term	What it is
Property class	Created at the IDO level; defines default attributes (length, data type) that properties inherit
Property	A named piece of info passed between client and database; can override its property class defaults
Property class extension	A global object overriding an IDO property class's attributes for *bound components specifically* — lets you standardize how UI overrides IDO behavior
Component class	A global object that packages a reusable bundle of component attributes (caption, list source, validator, etc.) — update the class, every inheriting component updates automatically
Component	The actual instance on the form (a textbox, combo box); can override everything above it

Components NOT bound to IDO properties only have two inheritance levels: Component Class → Component.

Practical rule of thumb: set up IDO property classes when developing IDOs; set up property class extensions only when components commonly need to override IDO property behavior; set up component classes whenever you want a reusable bundle. Following this means you rarely have to manually specify data type/format on every single component.

Manage via: `System > Edit > Component Class` (or Property Class Extension) in Design Mode — no form needs to be open.

Binding a component to an IDO property: Component sheet > Data Source > Binding. Bound components automatically gain the property's attributes. Two special behaviors:

- **Read-Only For Existing Records** — locks key fields once a record is created (default for key-bound components, but settable on any)
- **Read-Only For New Records** — opposite: locked only while creating, editable once saved

---

6. List sources

A **list source** populates list-type components (list box, drop list, combo box, enhanced combo box, grid column). Four types:

Type	How it's populated
IDO Collection	Queries a collection directly; first selected column always provides the component's value; supports filter/order-by/group-by/distinct via "Filter etc."
IDO Method	A stored-procedure method returns the value list; supports passed parameters
Inline	A hard-coded table of values you type directly into the list spec (Entries, Column for Value, Columns for Display)
Script	A global script populates the list via the system scripting API (covered only briefly)

Set up via Component sheet > Data Source > **List Source** ellipsis → **Edit List Source Specification** dialog.

---

7. Validators

Validators are reusable global objects that check a component's value, attached via Component sheet > Data Source > **Validation**.

Timing control — Validate Immediately:

- `True` = validates the instant the user tabs off the field
- `False` (default) = validates only at save or record-navigation time

Message display — Message on Status Bar: `True` = shows in status bar; `False` (default) = pop-up message box.

Substitution keywords for error messages: `%V` = current value, `%L` = component label, `%B` = message from a validation stored procedure.

Most common validator types: | Type | Checks | |---|---| | Alphanumeric | Rejects non-alphabetic/non-numeric characters; can add conditions like `V% > 31` | | DateTime | Valid date/time per locale; e.g. `V% <= CURDATE()` | | Inline list | Value must match one of the inline list's entries; can output other columns to a variable/property/component | | In Collection | Value must exist as a property value in a specified IDO's collection |

Manage globally via `System > Edit > Validators` even with no form open (New/Copy/Edit/Delete).

---

8. Strings

A **string** is a reusable global object holding translated text — used instead of hardcoded literal captions so one edit updates every form/component using it (and supports translation).

Naming prefix convention: | Prefix | Use | |---|---| | `s` | Static label captions (e.g. `sItem`) | | `f` | Form captions (e.g. `fCustomerOrders`) | | `m` | Messages (e.g. `mIsNotAValid`) | | `t` | Text (e.g. `tChangePOStatusUtility`) |

Create/copy/edit/delete via `System > Edit > String` — no form needs to be open. A Static component linked to another component's caption (using `C(<staticname>)`) inherits that component's caption automatically.

---

9. User-Extended Tables (UETs) and user-defined type lists

UETs let an admin add custom fields to existing application tables without modifying the base schema directly — useful for tracking data the standard app doesn't capture.

Workflow: UET Classes form (define a class name) → UET User Fields form (define fields + data type/length, e.g. `nvarchar(20)`) → UET Class/Field Relationships (link fields to class) → UET Table/Class Relationships (link class to a real table, e.g. `co_mst`) → **UET Impact Schema form** (click Process — this physically creates the database columns) → restart session to refresh cache.

If using replication, also click **Regenerate Replication Triggers** on the Replication Management form after any UET change.

User-defined types are reusable named lists of values (e.g. "Size" → Small/Medium/Large) usable as a list source without touching schema, IDOs, or code. New values can be added live via right-click "Add" once the type is in use. Particularly useful paired with UET fields — bind a combo box to the new UET property, then set its **Component Class to UserDefinedType** with the type name as the Parameter.

---

10. Event handlers

An **event handler** is a form-specific unit of logic that runs in response to a user (or programmatic) action — e.g. clicking a button, changing a combo box value, gaining/losing focus on a field.

Key mechanics:

- Identified by event name + **sequence number**; multiple handlers per event run in sequence order
- If a handler fails, by default later handlers in that sequence are **skipped** — override with the **Ignore Failure** parameter
- Built-in system responses for standard events always run **last** (after any custom handlers) — a custom handler reporting failure can cancel the built-in response
- Form events vs. application events exist; this lesson covers **form events only** (form-scoped, not shared across forms)

Main response type categories: Collection processing, Execute an executable, Form navigation, Generate event, Goto form, Inline script, Method call, Run background task, Run script, Set values. Also two named handler types used directly: **Goto URL** and **Run Form as Linked Child**.

Messages: define Error/Success messages per handler; can reference `%B` for stored-procedure messages; display in pop-up (default) or status bar (`Message on Status Line`). For multi-collection forms, scope a handler to one collection via **Only When Current Collection Is**.

Managing handlers: `Edit > Event Handlers` (Design Mode) → New/Copy/Edit/Delete, with Up/Down to resequence. Handlers inherited from a **base form** appear grayed out; to edit one, set **From Base Form** to False first (warning: this re-associates *all* handlers for that event with the extended form, not just the one you're editing).

---

11. FormSync — moving customizations between environments

The core problem it solves: you customize and test forms in a dev/test environment, then need those exact changes applied to production without manually redoing the work — and without losing your customizations the next time you apply an upgrade or service pack.

What FormSync does:

- Compares a source version against a target version + any existing target customizations
- Either merges (keeps customizations + applies new vendor data) or replaces, depending on your synchronization settings
- Replaces old vendor versions with new vendor versions in the target
- Can export customizations to XML files and import them elsewhere (the practical mechanism you'll likely use day-to-day)

Important limitations to know:

- **Any** attribute change counts as a "customization" for sync purposes — even a comment added to a script
- FormSync does **not** resolve layout conflicts (size/position) between your customization and a new base version — they can visually overlap after merge, requiring manual fixing
- **Scope: client tier only** (forms, global objects, WinStudio Design Mode changes). It does **not** touch IDO metadata, report definitions, table columns, stored procedures, Application Event System metadata, triggers, or functions — those need separate handling
- Old **VBA scripts** get auto-converted to VB.NET during sync (you're prompted to keep/remove/edit) — inline scripts are **not** auto-converted, just replaced with the new source version (you'd need to manually reapply)
- **Always test scripts after conversion** — no guarantee the converted code is fully functional

Practical export/import flow (what you'd actually do):

1. Open the **FormSync form**, set Source and Target (same instance for an export-only step)
2. Click **Utilities** → select scope type (e.g. Site) → Refresh List → select only the form(s) you want
3. **Export tab** → choose filename/path → Export → produces an XML file
4. On the target system: open FormSync → Utilities → **Import tab** → select the XML file → Import

---

Quick reference: Design Mode menu paths

Task	Path
Enter Design Mode	Toolbar button (after removing filter-in-place)
Change theme/fonts globally	`System > View > User Preferences > Runtime Layout`
Edit background color (reusable)	`System > Edit > Background Color`
Import an image	`System > Edit > Image`
Regenerate form	`System > Edit > Regenerate Form`
Revert form to previous version	`System > Edit > Revert Form Definition`
Copy a form	`System > Form > Definition > Copy`
Edit component classes	`System > Edit > Component Class`
Edit property class extensions	`System > Edit > Property Class Extension`
Edit validators	`System > Edit > Validators`
Edit strings	`System > Edit > String`
Edit event handlers	`Edit > Event Handlers`
Set tab order	`System > Edit > Component > Tab Order`
Check if form is customized	`System > Help > About This Form`
Check your permission level	`Help > About <servername>`

What's notably absent / out of scope for this workbook

This workbook does **not** cover: writing custom C# IDO methods from scratch, creating brand-new IDOs, SQL stored procedure development, or the WinStudio IDO editor in depth (Appendix B covers creating a *new form*, but not a new IDO). It also explicitly skips Tree and

User Control components as "beyond scope." If your actual work involves writing new business logic rather than personalizing existing forms, that's likely a separate, more advanced course/workbook — worth asking about specifically.